Natural Language Processing (NLP)

Preparing Text for Language Models

Data Input Pipelines for Language Models

Regex, Tokenization & Subword Encoding

By:

Dr. Zohair Ahmed

www.youtube.com/@ZohairAI Subscribe

www.begindiscovery.com

History of Computer Science

- Mathematics → Logic → Algorithms → Turing Machine → Real Computers → Software → AI
- Math invented logic + binary + algorithms
- Turing defined computation
- Engineers built electronic computers
- Software scientists built programming languages
- Internet connected the world
- AI + ML/NLP changed everything

First Phase

It all started with Mathematics (1800s)

- Before computers existed, the only way to think logically and solve problems was **mathematics**.
- Three major mathematicians shaped computer science:
- **George Boole (1847)**
  - Invented **Boolean Logic** (true/false)
  - This became the foundation of computer circuits
  - Without Boolean logic → no CPUs, no programming

Gottfried Leibniz (1700s)

- Invented **binary numbers (0 and 1)**
- Modern computers still use 0/1
- He imagined a machine that could compute using binary

Alan Turing (1936)

- Created the **Turing Machine**
- Showed that machines can follow algorithms
- Gave birth to the idea of "**computation**"

- **These were mathematical ideas, no real computers yet**

Intermediate Phases

2. Mechanical Computers (1800s → early 1900s)

- **Charles Babbage (1830s)**
  - Designed the **Analytical Engine**, a mechanical programmable machine
  - Ada Lovelace wrote the first **algorithm** for it
  - Many people call her **the first programmer**
- These machines were not electronic they were mechanical, but they followed the idea of instructions

3. Electronic Computers Begin (1930s–1940s)

- When electricity + circuits came, the mathematical ideas turned into real machines.
- **Key milestones:**
  - **1937:** Claude Shannon applies Boolean algebra to electrical circuits
  - **1941:** Konrad Zuse builds Z3 (first programmable computer)
  - **1945:** ENIAC created in USA (very big electronic computer)
- So mathematical logic finally became **real hardware**.

4. Birth of Modern Computer Science (1950s)

- Now computers existed but no programming languages, no operating systems.
- This decade introduced:
  - First programming languages: **FORTRAN**, **COBOL**
  - First compilers
  - First theories of algorithms
- This is the time when **Computer Science became a university subject**, separate from math.

Later Phases

5. Software Revolution (1960s–1980s)

- Computers became smaller and cheaper.
- Important developments:
  - C programming language
  - UNIX operating system
  - OOP (Object-Oriented Programming)
  - Internet beginnings (ARPANET)
- Now CS was no longer only math + circuits it became:
  - programming
  - networking
  - databases
  - software engineering

6. Internet + Personal Computers (1990s)

- This changed everything:
  - Web browsers → WWW boom
  - Windows OS
  - Linux
  - Email, web apps
  - Start of AI research using machine learning
  - People around the world started using computers.

7. Modern Era (2000s to now)

- Computer science exploded into many fields:
  - Artificial Intelligence
  - Machine Learning
  - NLP (your field!)
  - Deep Learning
  - Data Science
  - Cybersecurity
  - Cloud Computing
  - Mobile systems
  - Quantum computing
- Now CS is a giant field much bigger than mathematics.

1. Basic Text Processing

Text as Data: Characters & Unicode

Text in NLP = Just a Sequence of Characters

- Think of any text as a line of tiny building blocks called characters.
- Example: Cat → C a t
- Before NLP does anything smart (like sentiment analysis or classification), it first sees text as just characters in a row.

Tokens = Meaningful Chunks of Characters

- After characters, NLP usually breaks text into tokens, which are chunks like:
- words (Hello, world)
- subwords (un, finish, ed)
- sometimes even characters (for certain models)
- Tokenization helps models understand units of meaning instead of raw letters.

What is Unicode?

- Unicode = A Universal List of Characters
- Unicode is like a giant database that contains almost every character in every language:
  - English letters
  - Urdu letters
  - Chinese characters
  - Emojis
  - Punctuation
  - Symbols
- It gives each character a unique ID, so computers worldwide can agree on what a character is.
- 4. Code Points = Unique ID for Each Character
- A code point is just a number assigned to a character.
- Examples:
  - A → U+0041
  - ا → U+0627
  - 😊 → U+1F60A
- The “U+” simply means “Unicode”.

Encodings

- Encoding = Turning Characters into Bytes
- Computers store everything as bytes (0s and 1s).
- Encodings tell the computer how to convert Unicode code points into bytes.
- The most important encoding today is:
- UTF-8 (Universal + Efficient)
- Works for all languages
- Saves space
- Standard for the web and modern NLP
- Think of it as the packing method for Unicode characters.

Encodings: ASCII vs Unicode/UTF-8

- **ASCII**
  - Early 7-bit encoding (128 chars)
  - English-only: basic letters, digits, punctuation
- **Unicode + UTF-8**
  - Covers virtually all languages and symbols
  - UTF-8 is **variable-length**:
    a. 1–4 bytes per code point
    b. Backward-compatible with ASCII
- Rule: **Always know the file encoding**; default to UTF-8 when possible

Normalization

- Sometimes two pieces of text look the same on screen, but inside the computer they are stored differently.
- This causes problems in:
  - string comparison
  - searching
  - tokenization
  - deduplication
- So, normalization fixes this.
- **Example:** Same Visible Letter, Different Internal Forms
- Take the accented letter: **é**
- There are two ways to store it:

1. Precomposed (single character)

- A ready-made character: é → U+00E9

2. Decomposed (base + accent)

- Stored as two characters:
- **e** → U+0065
- ´ → U+0301 (combining accent)
- On the screen → looks identical
- Inside the computer → different bytes
- So, the computer sees them as not equal unless normalized.

What is Unicode Normalization?

- Normalization = cleaning the text so all variants become one standard form.
- It converts text into a canonical (consistent) structure.
- Two Common Forms

1. NFC (Normalization Form Composed)

- Uses precomposed characters
- Single code point if possible
- Example: é → (U+00E9)

2. NFD (Normalization Form Decomposed)

- Uses base + combining marks
- Multiple code points
- Example: e + ´ → (U+0065 + U+0301)
- Both look exactly the same to humans.

When Should You Normalize?

- Normalize BEFORE doing:

1. String comparison

- "é" == "é" # may be false if not normalized

2. Deduplication

- Two same-looking names may be treated as different.

3. Tokenization / regex (Pattern matching tool for searching text)

- Especially in:
- Urdu
- Arabic
- Hindi
- Any script using combining marks

4. Cleaning NLP datasets

- Avoid hidden inconsistencies.

Lowercasing vs Case-Sensitive Modeling

Lowercasing: What & Why?

- Lowercasing = converting all text to small letters.
- **Example:** "Apple IS GREAT" → "apple is great"

Advantages of Lowercasing

- **1. Reduces vocabulary size**
- "Apple", "APPLE", "apple" become one word.
  - Smaller vocabulary
  - easier training
  - fewer parameters
  - fewer rare words
- **2. Helpful for noisy or informal text**
- Social media, chats, comments often mix cases:
- "LOL" = "lol"
- "hELp" → "help"
- Lowercasing makes all of them consistent.

Case-Sensitive: What & Why?

- Case-sensitive models keep the original uppercase and lowercase letters.
- Example: "Apple" ≠ "apple"

Advantages of Case-Sensitive Text

- **1. Preserves important meaning**
  - Apple = company
  - apple = fruit
- **2. Keeps sentence information**
- Capital letters show sentence starts:
- "This is good. That is bad."
- **3. Useful for acronyms/ abbreviation**
- "USA" vs "usa", "AI" vs "ai" (sometimes treated differently)
- Case helps models learn these distinctions.

Typical Practice (What NLP usually do)

Old or classic NLP (like n-gram models):

- Often converted everything to lowercase
- Because these models struggled with large vocabularies

Modern NLP (BERT, GPT, XLM-R, etc.):

- Usually case-sensitive
- Use subword tokenization, which handles:
- upper/lowercase
- rare words
- multilingual text
- So, they keep "Apple" different from "apple" when useful.

Handling Punctuation, Numbers, URLs

Punctuation

- Helps models understand **sentence boundaries**
  - Example: ".", "?", "!"
- Also marks **clauses** and **lists**
- Tokenizers usually **split them out** as separate tokens
  - "Hello!" → "Hello", "!"

Numbers

- **Keep full number as is**
  - "2026" stays "2026"
- **Normalize numbers** (very common)
  - "2026" → <NUM>
  - "145,000" → <NUM>
- → Good for reducing vocabulary size.
- **Break into digit chunks** (subword tokenizers)
  - "2026" → "20", "26" or "2", "0", "2", "6"

URLs / Emails / Handles / Hashtags

- Often treated as **one whole token**, because splitting them loses meaning.
- Examples:
- "https://site.com" → <URL>
- "abc@gmail.com" → <EMAIL>
- "@username" → <HANDLE>
- "#AI2026" → "#AI2026" or <HASHTAG>

Why normalize?

- URLs/emails are long
- Contain many special characters
- Don’t add semantic meaning

Handling Emojis, HTML

Emojis

- Emojis are **Unicode characters**
- Carry **strong emotion/sentiment**
  - 🙂 positive
  - 😡 negative
  - 💔 sadness
- Tokenizers usually keep them **as single tokens**
- Emojis act like **sentiment words**.

HTML & Markup

- Found in web-scraped text.
- **Option 1: Strip tags (most common)**
- Hello World → Hello World
- **Option 2: Keep structure only if needed**
- Useful for:
- web-page layout analysis
- document formatting tasks
- Most NLP tasks → **just remove HTML**.

2. Regular Expressions

What Is a “Formal Language”?

- A **formal language** is: A set of rules that tell how valid strings (patterns) are formed.
- In simple words:
- A **formal language = rules for building strings**
- Not a natural language (like Urdu or English)
- It’s a **mathematical language**
- **Example:**
- If I make a formal language with rules:
- A string must start with a
- Then have 1 or more b’s
- End with c
- This creates a “formal language” with strings like:
- abc
- abbc
- abbbbc
- Regex is based on these kinds of mathematical rules.

Why Was Regex Made?

- Regex was created to solve one big problem:
- **How can we describe patterns in text using math?**
- Examples of patterns:
- “any number”
- “any word ending with ‘ing’”
- “any email address”
- “any string with 3 digits”
- Before regex existed, computers could **only match exact words**, not patterns.
- So the creators wanted a way to:
- **search smarter**
- **match flexible patterns**
- **analyze text automatically**
- **build compilers**
- **process human language**
- Regex gave computers a **powerful and compact way** to describe text patterns.

Regular Expressions

- Regex = formal language to specify sets of strings
- Core idea: match **patterns**, not just fixed substrings

1. Pre-tokenization

- Split text into words, punctuation, numbers, emojis, etc.
  Example: split on whitespace, punctuation, digits.

2. Cleaning & normalization

  - Remove extra spaces
  - Remove HTML tags
  - Replace numbers/emails/URLs

- **Example:**
  \d+ matches digits

3. Extracting structured fields

- Useful when text contains information like:
  - dates
  - prices
  - phone numbers
  - email addresses
- Regex can automatically pull these out from messy text.
- **Example:**
- Match *any* number → \d+
- Match *any* email → \S+@\S+
- Match *any* word starting with "\" → \\w+
- So instead of searching "2026" exactly,
  regex finds **any 4-digit number**.

Basic Regex Syntax I: Literals & Character Classes

- **Literals**
  - Exact characters that regex should match.
  - cat matches the exact string “cat”
- **Character classes**
  - [abc] : one of a, b, c
  - [0-9] : any digit
  - [A-Za-z] : any ASCII letter
  - [^...] : negation (e.g. [^0-9] = non-digit)
- **Predefined classes** (common)
  - \d : digit [0-9]
  - \D : non-digit
  - \w: = [A-Za-z0-9_]
  - \W : non-word char
  - \s : whitespace (space, tab, newline)
  - \S : non-whitespace

Basic Regex Syntax II: Quantifiers & Grouping

- **Quantifiers**
- Quantifiers tell regex “how many times” something should repeat, Think of them as counters.
  - * : 0 or more
  - + : 1 or more
  - ? : 0 or 1 (optional)
  - {n} : exactly n
  - {n,} : n or more
  - {n,m} : between n and m

- **Grouping**
  - (...) : groups pattern
  - | : alternation (OR), e.g. cat|dog
  - (ab|cd) : matches “ab” or “cd”

Anchors & Boundaries

- **Anchors do NOT match characters.**
- They match positions in a string
  - **^ :** start of string (or line)
  - **$:** end of string (or line)
- Boundaries check a word boundary (between word and non-word characters):
- **\b : Word Boundary**
- This means: “Position between **word** characters and **non-word** characters.”
- Example: \bcat\b
- Matches:
  - cat
- Not matches:
  - concatenate
  - educator
  - bobcat
- Because in those, **cat** is inside another word, not a boundary.
- **\B : NOT a Word Boundary**
- Opposite of \b. Matches where **there is no boundary**.
- Example:
  - \Bcat\B
- Matches:
  - concatenate
  - educator
  - bobcats
  - Not match:
  - cat
- Because there *are* boundaries around standalone “cat”.

Cleaning Logs / Text with Regex

- Remove or normalize noise:
  - Timestamps
  - IP addresses
  - Log levels (INFO, ERROR)
- Examples (conceptual):
  - Replace multiple spaces by a single space:
    a. Pattern: \s+ → " "
  - Remove HTML tags:
    a. Pattern: <[^>]+> → ""
  - Strip non-text control sequences (as needed for your corpus)

Extracting Structured Info with Regex

- Typical extraction tasks:
  - **Dates**
    a. E.g. \d{4}-\d{2}-\d{2} (YYYY-MM-DD)
  - **Email addresses**
    a. E.g. [A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}
  - **Phone numbers**
    a. Country/locale-specific patterns
- Use capturing groups:
  - (...) to extract subparts (day, month, year, area code, etc.)
- Pipeline:
  - Apply regex → collect matches → convert to structured fields

3. Words, Corpora, Morphology

Types vs Tokens & Vocabulary

- **Token**
  - Single occurrence in text
  - “the dog barks” → 3 tokens
- **Type**
  - A **type** is a **unique word form** in the vocabulary of a corpus.
  - From the corpus:
  - **“the dog barks”** → 3 types
  - “the”
  - “dog”
  - “barks”
- If the sentence repeats 10 times, you still have **3 types**, because types count *unique words*, not occurrences.
- **Vocabulary**
  - The vocabulary of a corpus = the set of all unique word types it contains.
  - As you add more text to a corpus, the number of unique word types (vocabulary size) grows.

Types vs Tokens & Vocabulary

Heaps' Law (Vocabulary Growth Law)

- Heaps' Law describes how vocabulary size increases as corpus size increases.
- Formal form: $V(N) = KN^{\beta}$
- **V(N)** = vocabulary size (number of unique types)
- **N** = number of tokens in the corpus
- **K, β** = constants (β usually between 0.4–0.6)

Types vs Tokens & Vocabulary

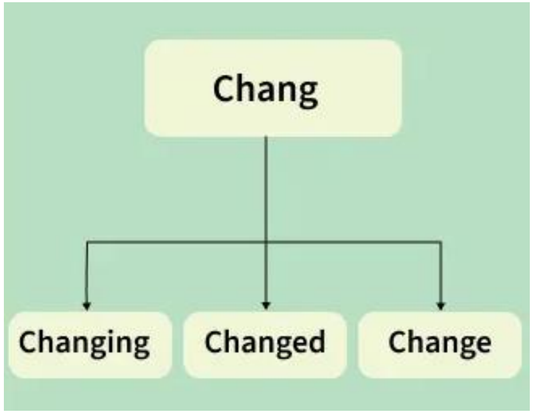
- **What does "Grows with corpus size (Heaps' Law)" mean?**
- As you add more text:
  - you get more **tokens**
  - and **new word types keep appearing**
- Even if you're reading millions of words, new words still show up occasionally.
- **The key implication: Vocabulary is effectively *unbounded* in real-world language data.**
- As corpora get larger, vocabulary keeps increasing.
- You never reach a fixed, closed list of all words.
- Natural language constantly introduces names, slang, spellings, compounds, etc.
- **Example:** If you collect 1 million tokens and then 2 million more, you will still find **thousands** of new unique words.

This has big implications for:

- building dictionaries
- language models
- handling out-of-vocabulary (OOV) words
- tokenization (subword methods like BPE)

Stemming

- **Stemming** and **Lemmatization** reduce words to a simpler or base form, but they differ in accuracy, linguistic depth, and use cases.
- **1. Stemming**
- A stemmer chops off word endings using simple rules (heuristics).
- It does not use a vocabulary or grammar rules.
- **Example**
  - running → run
  - runs → run
  - universities → univers



- You can see the last example produces non-words, this is common.
- **Key Characteristics**
  - Very fast, simple rule-based chopping.
  - May produce incorrect, non-dictionary stems.
  - Not linguistically informed.

Use Cases (when to use stemming)

- Search engines / Information Retrieval (IR)
- Large-scale text mining where speed > precision
- When exact linguistic correctness is not required

Stemming

(A) Porter Stemmer: One of the oldest and most widely used English stemmers.

- Invented by Martin Porter (1980).
- Uses a series of 5 rule-based steps with dozens of conditions.
- Strong focus on suffix stripping:
  - caresses → caress
  - ponies → poni
  - caress → caress (unchanged)

Advantages

- Simple, efficient, widely implemented.
- Good balance between speed and moderate accuracy.

Disadvantages

- Sometimes under-stems or over-stems, e.g.
  - universal → univers
  - relational → relat

(B) Snowball Stemmer (aka Porter2)

- An improved version of the Porter stemmer.
- Also created by Martin Porter.
- Rewritten in the Snowball language (a language for writing stemmers).
- More consistent, clearer rules, and better performance.

Improvements over Porter

- Handles more English suffix variations
- Better consistency
- Fewer overly-aggressive stems
- More maintainable rule-set

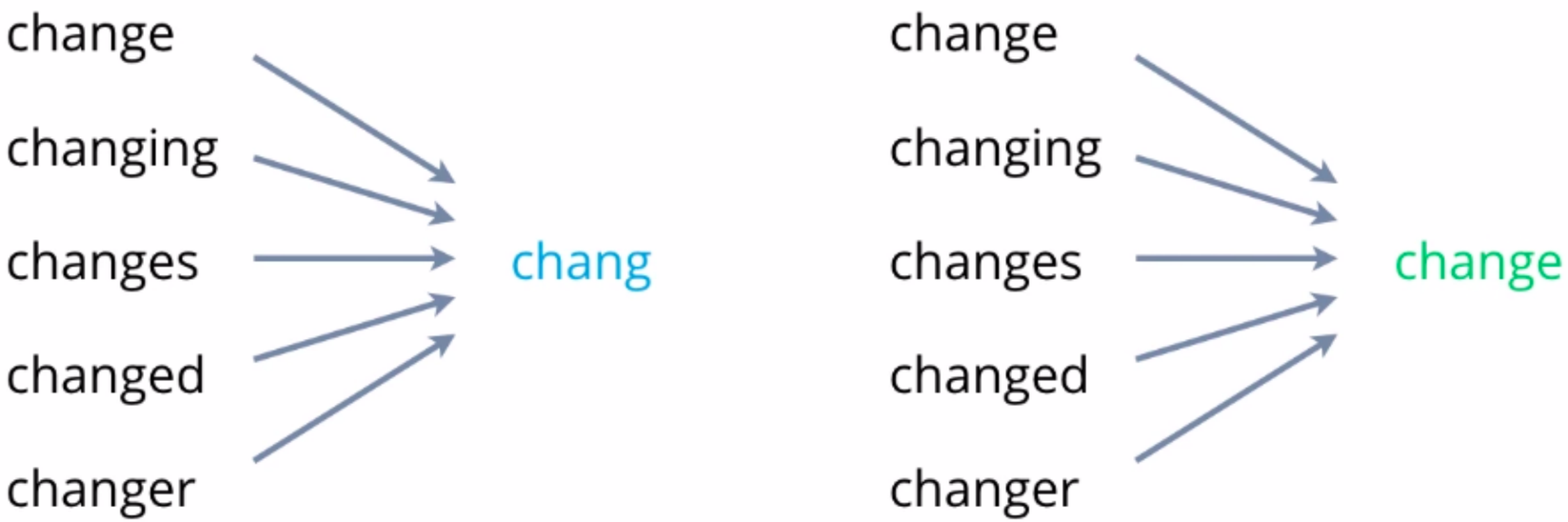
Example

- running → run
- happiness → happi
- generalized → general

Other approaches like Lancaster Stemmer

Lemmatization

Stemming vs Lemmatization



Lemmatization

2. Lemmatization

- A lemmatizer uses:
  - A vocabulary (dictionary)
  - Morphological analysis
  - Part-of-speech (POS) information (often)
- It returns the actual dictionary form (lemma).
- Examples
  - better → good
  - ran → run
  - cars → car

Key Characteristics

- More accurate, linguistically correct
- Slower (because it uses dictionaries & morphology)
- Produces valid dictionary words

Use Cases (when to use lemmatization)

- Semantic analysis
- Machine learning models
- NLP pipelines requiring linguistic precision
- Topic modeling, text analytics
- Tasks where meaning matters

Lemmatization

(A) Rule-based Lemmatization

- Uses grammar rules + vocabulary.
- Example: WordNet Lemmatizer (in NLTK).

Steps:

- Part-of-speech tagging
- Dictionary lookup
- Morphological rules applied

(B) Dictionary-based Lemmatization

- Specialized lexicons map modified forms to lemmas.
- Example:
  - ran → run
  - better → good
  - children → child
- **(C) Machine Learning-based Lemmatization**
- ML models learn morphological patterns, often for complex languages.
- **Approaches:**
  - Classification models
  - Seq2Seq models
  - Transformer-based lemmatizers
  - Neural morphological analyzers
- Especially useful for:
  - Urdu
  - Arabic
  - Finnish
  - Turkish
  - Where morphology is richer.

Morphology: Inflectional vs Derivational

A **morpheme** is the smallest unit of language that has meaning.

- Think of it like the Lego bricks of words.
  - cat (has meaning)
  - -s (means plural)
  - walk (action)
  - -ed (past tense)
- You can't break a morpheme into smaller meaningful parts.

Inflection changes the grammatical form of a word, but **NOT its basic meaning**.

- Keeps the same word, Adds information like:
  - plural (cat → cats)
  - past tense (walk → walked)
  - subject–verb agreement (walk → walks)
- Examples:
  - **cat → cats** (same word "cat," but now plural)
  - **walk → walked** (same word "walk," just past tense)
- You don't get a *new* dictionary word, just a *modified version*.

Derivational Morphology

- **Derivation** creates a **new word** (a new lexeme), often changing the **word class**.
- Often changes meaning
- Often changes part of speech (noun → verb, adj → noun, etc.)
- Examples:
  - **happy → happiness** (adjective → noun)
  - **modern → modernize** (adjective → verb)
- Here, the new words "happiness" or "modernize" have their own meaning and can be found independently in a dictionary.

Morphology and Tokenization

- In NLP, tokenization sometimes splits words into smaller pieces. These pieces may correspond to **morphemes**.
- For example:
  - "walked" may be tokenized as **walk + ed**
  - "unhappiness" may be **un + happi + ness**
- This shows how tokenizers must deal with morphemes (roots, prefixes, suffixes).

Languages & Morphological Typology

1. Isolating Languages

- These languages use **very few morphemes per word**.
  Often, **one word = one morpheme**.
- **Examples**
  - **Vietnamese**
  - **Chinese**
- In these languages:
- Words don't usually have prefixes/suffixes.
- Meaning is expressed through *separate words*, not word endings.
- **So, tokenization is easy**: Word = Token (most of the time)

2. Synthetic / Polysynthetic Languages

- These languages pack **many morphemes inside one word**.
- **Examples**
  - **Turkish** (agglutinative)
  - **Koryak** (polysynthetic)
- In such languages:
- A single word can contain **root + many affixes**.
- A word might express what **a whole sentence** expresses in English.
- Example (Turkish):
  - **evlerinizden** = "from your houses"
  - Contains multiple morphemes: *ev* + *ler* + *iniz* + *den*
- **One word ≠ one meaning**
  One word may contain multiple meaningful chunks

3. Why This Matters for NLP Tokenization

- **Word-based tokenization may fail**
- If we split text into tokens only by whitespace:
- "evlerinizden" is treated as *one token*,
- even though it contains **four morphemes** important for meaning.
- This leads to:
  - Data sparsity
  - Worse language modeling
  - Poor generalization

4. Why We Need Subword Tokenization

- To handle rich morphology, NLP uses **subword tokenization**, such as:
  - BPE (Byte-Pair Encoding)
  - WordPiece
  - SentencePiece
- These methods break words into **smaller units**, often aligned with morphemes.
- Example using subwords:
  ev + ler + iniz + den
- Benefits:
  - Better handles long, complex words
  - Reduces vocabulary size
  - Helps for unseen words (OOV words)

Tokenization & Sentence Segmentation

- Tokenization: split running text into **tokens**
- Candidate token units:
  - Words
  - Subwords
  - Characters
- Goals of Tokenization

Standard, deterministic representation

- Same text input → same token output
- **Helps ensure consistency across:**
  - Training
  - Testing
  - Production systems

Enable consistent length measures

- Counting tokens instead of characters or words
- Useful for:
  - Model input size
  - Sequence length control
  - Budgeting (e.g., LLM token cost)

Proper model inputs

- Most NLP models require text to be converted into tokens
- Tokens → IDs → embeddings → fed into model

Word-Level Tokenization: Challenges

- **Punctuation**
  - U.S.A., Ph.D., costs $12.40...
- **Clitics & Contractions**
  - English: don't, we're, they're
  - French: l'homme, j'ai
- **Multiword expressions**
  - New York, rock 'n' roll – sometimes treated as single units
- **Languages without spaces**
  - Chinese, Japanese, Thai require word segmentation
- **Social media / code-switching**
  - Non-standard spellings, mixed languages

Rule-Based Word Tokenization

- Define target **standard** (e.g. Penn Treebank)
  - Separate most punctuation
  - Keep U.S.A., Ph.D. intact
  - Separate clitics: doesn't → does + n't
  - Keep hyphenated words together: poster-print
  - Preserve numbers and currency: $12.40, 555,500.50
- Implement via:
  - Carefully designed regular expressions
  - Finite-state tokenizers (e.g. NLTK regex tokenizer)

- **Example output:**
  - Input: That U.S.A. poster-print costs $12.40...
  - Tokens: That, U.S.A., poster-print, costs, $12.40, ...
- **Typical rules:**
  - Abbreviations: (?:[A-Z]\.)+
  - Words with internal hyphens: \w+(?:-\w+)*
  - Currency/percentages: \$?\d+(?:\.\d+)?%?
  - Ellipsis: \.\.\.
  - Punctuation: [.,;\"'?():_\[\]-]

Sentence Segmentation

- **Task:** split text into **sentences**
- **Signals in written English:**
  - Sentence-final punctuation: ., ?, !
  - But: period . is ambiguous:
    a. Abbreviations (Dr., Inc.)
    b. Sentence-final abbreviation (Inc. at end)
- **Typical rule-based approach:**
  - Decide if . is end-of-sentence or part of a token
  - Use abbreviation dictionaries and context rules
- **In many NLP toolkits:**
- Sentence boundaries are a deterministic function of tokenization.
  - First, the text is tokenized.
  - Then rules look at token sequences to decide where sentences end.
- Example:
- If tokenizer labels “Dr.” as one token, rules know it is an abbreviation → don’t break a sentence there.

Rule-Based vs ML-Based Sentence Splitting

- **Rule-based**
  - Deterministic
  - Fast, easy to explain
  - Needs curated abbreviation lists and patterns
- **ML-based**
  - Treat boundary detection as classification
  - Features: punctuation, surrounding tokens, capitalization, abbreviations
  - Better for noisy or multilingual text
- Many production pipelines: rule-based + heuristics is still common.

NLP Example

- **Deterministic:**
  - A predefined abbreviation list:
- "Dr." → never end of sentence.
- **Heuristic:**
  - If period + capital letter → probably end of sentence
  - If token is short (1–2 letters) and ends with a period → maybe abbreviation (e.g., "U.S.")

Motivation for Subword Tokenization

- Problems with **word-level vocabularies**:
  - Out-of-vocabulary (OOV) words
  - Vocabulary explosion (names, misspellings, compounds)
  - Poor handling of morphology and rare forms
- **Subword approach:**
  - Learn units between characters and words
  - Compose any new word from known subwords
- **Benefits:**
  - No OOVs (in practice)
  - Better parameter sharing across related word forms
  - Control over vocabulary size vs sequence length

Design Space: Tokens & Vocabulary

Characters

- Every character = one token
  - Example: “cat” → ["c", "a", "t"]

Pros:

- Very small vocabulary (e.g., 100–500 tokens)
- Can handle any word, any language
- Great for morphologically rich languages (Turkish, Koryak, Urdu)

Cons:

- Very long sequences
- “catastrophic” becomes 12 tokens
- Slower training
- Harder for model to learn long dependencies

Design Space: Tokens & Vocabulary

Whole Words

- Example:
  - “unhappiness” = 1 token
- **Pros:**
  - Shortest sequences
  - Very intuitive
  - Easy to interpret
- **Cons:**
  - Huge vocabulary (hundreds of thousands)
  - Explosion of rare words
  - Fails badly in morphologically rich languages
  - Cannot handle new/unknown words (OOV problem)

Byte-Pair Encoding (BPE): Intuition

- Originally a compression algorithm; adapted for NLP
- Neural Machine Translation of Rare Words with Subword Units, (Sennrich et al., 2016).
  - https://aclanthology.org/P16-1162/
- Basic idea:
  - Start with characters as tokens
  - Iteratively merge the **most frequent adjacent pair** of tokens
  - Add merged pair to vocabulary as a new token
- Resulting vocabulary:
  - Characters + frequent subwords + some whole words
  - Uncommon words split into multiple tokens

Algorithm 1 Learn BPE operations

```
import re, collections

def get_stats(vocab):
  pairs = collections.defaultdict(int)
  for word, freq in vocab.items():
    symbols = word.split()
    for i in range(len(symbols)-1):
      pairs[symbols[i],symbols[i+1]] += freq
  return pairs

def merge_vocab(pair, v_in):
  v_out = {}
  bigram = re.escape(' '.join(pair))
  p = re.compile(r'(?<!\S)' + bigram + r'(?!\S)')
  for word in v_in:
    w_out = p.sub(''.join(pair), word)
    v_out[w_out] = v_in[word]
  return v_out

vocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,
         'n e w e s t </w>':6, 'w i d e s t </w>':3}
num_merges = 10
for i in range(num_merges):
  pairs = get_stats(vocab)
  best = max(pairs, key=pairs.get)
  vocab = merge_vocab(best, vocab)
  print(best)
```

BPE Training Procedure

- **Input:** training corpus with approximate word boundaries
- **Steps:**
  - Initialize vocabulary = all characters (optionally plus space symbol)
  - Count frequencies of all adjacent token pairs
  - Merge the most frequent pair into a new token
  - Replace all occurrences of that pair in the corpus
  - Repeat steps 2–4 for k merges
- Final vocabulary:
  - Size = number of characters + k
  - k is hyperparameter controlling vocabulary size

BPE Encoding (Test Time)

- **Given:**
  - Learned list of merges (in order)
- **Encoding a new sentence:**
  - Pre-tokenize (e.g. by spaces/punctuation) into word-like strings
  - Split each into characters
  - Apply merges **greedily in learned order**
  - Stop when no more merges apply
- **Properties:**
  - Frequent words become single tokens
  - Rare/novel words decomposed into known subwords or characters

BPE in Practice: UTF-8 & Pre-tokenization

- Modern BPE variants often:
  - Run on **UTF-8 bytes** rather than code points
  - Use regex-based **pre-tokenization**:
    a. Split on whitespace and punctuation
    b. Optionally treat contractions and digits specially
- Example pre-tokenization strategy:
  - Separate contractions ('s, 're, 've, 'll, etc.)
  - Group sequences of letters as one token
  - Group sequences of digits as one token
  - Treat sequences of non-alphanumeric symbols as tokens
  - Preserve or normalize whitespace

WordPiece: Likelihood-Based Subword Merging

- Used in models like BERT
- Similar goals to BPE; key difference = **objective**
- **Procedure (conceptual):**
  - Start from characters
  - Iteratively add new subwords that **increase likelihood** under a language model
  - Stop when reaching target vocabulary size (e.g. 30k)
- **Characteristics:**
  - Usually trained over word-level boundaries (no cross-word merges)
  - Vocabulary size given; algorithm chooses best units

SentencePiece / Unigram LM Tokenization

- Often referred to as **SentencePiece Unigram LM**
- Approach:
  - Start from a **very large candidate vocabulary**:
    a. All characters
    b. Frequent words and substrings
  - Train a **unigram LM** over tokens
  - Iteratively:
    a. Estimate token probabilities
    b. Remove low-utility tokens
    c. Retain those that appear in high-probability segmentations
  - Stop when target vocabulary size is reached
- Properties:
  - Naturally works on raw text (no pre-tokenization required)
  - Tends to learn more semantically meaningful subwords than BPE

Comparing BPE vs WordPiece vs Unigram

- **BPE**
  - Greedy, frequency-based merges
  - Simple, widely used (e.g., GPT-style models)
  - May over-fragment some languages, create less meaningful pieces
- **WordPiece**
  - Maximizes sequence likelihood under n-gram LM
  - Fits well with encoder-style models (BERT)
- **Unigram / SentencePiece**
  - Probabilistic model over segmentations
  - Removes rather than merges
  - Often yields cleaner morphological units

Vocabulary Size & Sequence Length

- Larger vocabulary:
  - Fewer tokens per sentence
  - More memory for embeddings & softmax
- Smaller vocabulary:
  - More tokens per sentence
  - More steps per forward pass, higher compute
- Multilingual tokenizers:
  - English often dominates vocab in web-scale data
  - Other languages can get **over-segmented** (many tokens per word)
  - Leads to:
    a. Longer sequences
    b. Potentially worse representations and higher cost

Putting It Together: Tokenization Stack

- Typical modern pipeline:
  - **Basic text processing**
    a. Encoding → Unicode normalization → cleanup
  - **Pre-tokenization (optional)**
    a. Regex-based splitting (spaces, punctuation, clitics, digits)
  - **Subword tokenizer**
    a. BPE / WordPiece / Unigram LM
  - **Sentence segmentation** (if needed)
    a. Rule-based or ML-based
- Consistent tokenization is critical:
  - Reproducible experiments
  - Comparable metrics (e.g. perplexity, length)

Putting It Together: Tokenization Stack

- Typical modern pipeline:
  - **Basic text processing**
    a. Encoding → Unicode normalization → cleanup
  - **Pre-tokenization (optional)**
    a. Regex-based splitting (spaces, punctuation, clitics, digits)
  - **Subword tokenizer**
    a. BPE / WordPiece / Unigram LM
  - **Sentence segmentation** (if needed)
    a. Rule-based or ML-based
- Consistent tokenization is critical:
  - Reproducible experiments
  - Comparable metrics (e.g. perplexity, length)

Putting It Together: Tokenization Stack

- Follow Given Google Colab

- References
- https://sebastianraschka.com/blog/2025/bpe-from-scratch.html